

Customer Support Tickets Analytics Project

Project Title

Customer Support Tickets Classification and Analytics using Machine Learning

Skills Takeaway From This Project

- **Data Preprocessing & Cleaning:** Handling missing values, data type conversions, and feature engineering
- **Exploratory Data Analysis (EDA):** Understanding data distributions, patterns, and correlations
- **Feature Engineering:** Creating meaningful features from customer support data
- **NLP Text Processing:** Text cleaning, tokenization, TF-IDF, and word embeddings
- **Deep Learning for NLP:** RNN, LSTM, GRU, and Transformer models for text classification
- **Text Embeddings:** Word2Vec, GloVe, BERT, and pre-trained language models
- **Sequence Models:** Building and training RNN-based models for sequential data
- **Imbalanced Dataset Handling:** Techniques to address class imbalance (SMOTE, oversampling, undersampling)
- **Classification Models:** Building and evaluating ML and deep learning models for ticket classification
- **Regression Models:** Predicting continuous values like resolution time and satisfaction scores
- **Model Evaluation:** Using appropriate metrics for classification and regression tasks
- **Model Interpretation:** Understanding feature importance and model decisions
- **End-to-End ML Pipeline:** From data ingestion to model deployment
- **Customer Support Domain Knowledge:** Understanding ticket categories, SLA metrics, and customer segmentation
- **Data Visualization:** Creating meaningful visualizations to present insights
- **Streamlit UI:** Building interactive dashboards for ticket analysis

Domain

Customer Support & Service Management

Problem Statement

Objective: Build a machine learning system to automatically classify customer support tickets into priority levels using deep learning models.

Specific Tasks:

1. **Multi-class Classification using Deep Learning:** Develop a deep learning model (RNN, LSTM, GRU, or Transformer) to classify customer support tickets into priority levels (Urgent, High, Medium, Low)
2. **Model Comparison:** Compare deep learning performance with traditional ML models

Key Requirements:

- **Mandatory:** Use deep learning models (RNN, LSTM, GRU, or Transformer/BERT) for priority classification
- **Mandatory:** Implement text embeddings (Word2Vec, GloVe, or BERT) for NLP tasks
- Achieve **80%+ accuracy** for priority classification using deep learning
- Handle imbalanced dataset (urgent tickets are typically fewer than low priority ones)
- Process and analyze text fields (issue_description, resolution_notes) using NLP techniques and sequence models

Deliverable: A complete end-to-end deep learning pipeline that:

- Takes raw customer support tickets data as input from `customer_support_ticket.csv`
- Uses train/validation/test splits for model evaluation
- Preprocesses text data and creates embeddings
- Implements RNN/LSTM/Transformer models for priority classification
- Provides comparison between deep learning and traditional ML models

Alternative Problem Statements

Note: Students must complete **at least 2** of the following problem statements (including the original ticket classification). This ensures comprehensive learning across different ML techniques.

Option 1: Resolution Time Prediction (Regression with Neural Networks)

Objective: Build a regression model using neural networks to predict how long it will take to resolve customer support tickets.

Specific Tasks:

1. **Regression Modeling with Deep Learning:** Develop a neural network (ANN/Dense layers) to predict the exact resolution time in hours
2. **Time Analysis:** Understand which factors (priority, category, customer segment, channel) most influence resolution time
3. **SLA Optimization:** Identify factors that lead to SLA breaches

Key Requirements:

- Achieve **R² score of 0.70+** on test set
- **Mandatory:** Use neural networks for the regression task
- Perform feature importance analysis to identify resolution time drivers
- Handle outliers in resolution times appropriately
- Create visualizations showing time distributions by priority, category, and region

Approach:

- Use `customer_support_ticket.csv` with train/validation/test splits
- EDA: Analyze resolution time distribution, identify outliers
- Feature Engineering: Create time-based features, encode categorical variables
- Models: **Neural Networks (MLP/Dense layers)** with comparison to baseline models (Linear Regression, Random Forest)
- Evaluation: R², RMSE, MAE metrics
- Interpretation: SHAP values to understand feature impact on resolution time

Deliverable: A neural network regression model that can predict ticket resolution time with explanation of key drivers.

Option 2: Customer Satisfaction Prediction (Classification or Regression)

Objective: Predict customer satisfaction scores based on ticket characteristics and resolution details.

Specific Tasks:

1. **Classification/Regression:** Predict customer satisfaction score (1-5 scale)
2. **Satisfaction Analysis:** Identify factors that most impact customer satisfaction
3. **Improvement Recommendations:** Suggest actions to improve customer satisfaction

Key Requirements:

- Achieve **75%+ accuracy** for satisfaction classification (or R² 0.65+ for regression)
- Identify key satisfaction drivers (response time, resolution quality, channel)
- Handle class imbalance in satisfaction scores
- Create visualizations showing satisfaction distributions across customer segments

Approach:

- Use `customer_support_ticket.csv` with train/validation/test splits
- EDA: Analyze satisfaction score distribution, identify patterns
- Feature Engineering: Create interaction features, encode categorical variables
- Models: Logistic Regression, Random Forest, XGBoost, **Neural Networks**

- Evaluation: Accuracy, F1-Score, RMSE, MAE metrics
- Interpretation: Feature importance, SHAP values for satisfaction drivers

Deliverable: A model that can predict customer satisfaction with explanation of key factors.

Option 3: SLA Breach Prediction (Classification)

Objective: Build a classification model to predict whether a ticket will breach its Service Level Agreement (SLA).

Specific Tasks:

1. **Binary Classification:** Predict SLA breach (Yes/No) for incoming tickets
2. **Risk Assessment:** Identify high-risk tickets likely to breach SLA
3. **Prioritization System:** Recommend ticket prioritization to avoid SLA breaches

Key Requirements:

- Achieve **85%+ accuracy** for SLA breach prediction
- Keep false positive rate below **15%**
- Provide interpretable reasons for SLA breach predictions
- Create a scoring system to rank tickets by SLA breach risk

Approach:

- Use `customer_support_ticket.csv` with train/validation/test splits
- EDA: Analyze SLA breach patterns by priority, category, and region
- Feature Engineering: Create time-based features, encode categorical variables
- Models: Logistic Regression, Random Forest, XGBoost, Gradient Boosting, **Neural Networks**
- Evaluation: Accuracy, Precision, Recall, F1-Score, ROC-AUC
- Interpretation: Feature importance, SHAP values for SLA breach factors

Deliverable: A classification model (including neural network) that predicts SLA breach risk with explanations.

Business Use Cases

1. **Customer Support Teams:** Automate ticket triage and routing to improve response times

2. **Product Teams:** Identify common issues and feature requests to prioritize product development
3. **Management:** Monitor support performance metrics and SLA compliance
4. **Quality Assurance:** Identify patterns in customer dissatisfaction and resolution quality
5. **Resource Planning:** Optimize support team allocation based on predicted ticket volume and complexity
6. **Customer Experience:** Proactively identify at-risk customers and prevent churn

Approach

Phase 1: Data Understanding & Exploration

- Load and examine the dataset structure using pandas
- Understand each column and its significance:
 - ticket_id: Unique identifier (not useful for modeling)
 - customer_name/email: Customer identifiers (use for aggregation, not direct modeling)
 - product: Categorical feature (Web Portal, Mobile App, Payment Gateway, etc.)
 - category: Target variable for classification
 - issue_description: Text field for NLP processing
 - resolution_notes: Text field for NLP processing
 - priority: Target variable for priority classification
 - status: Ticket status (Open, Closed, Pending Customer, In Progress, Resolved)
 - channel: Communication channel (Email, Chat, Social Media, Phone)
 - region: Geographic region
 - customer_age: Numerical feature, check for outliers
 - customer_gender: Categorical, needs encoding
 - subscription_type: Categorical (Free, Premium, Basic, Enterprise)
 - customer_tenure_months: Numerical feature
 - previous_tickets: Numerical feature
 - customer_satisfaction_score: Target for regression/classification
 - first_response_time_hours: Numerical feature
 - resolution_time_hours: Target for regression
 - ticket_created_date/resolved_date: Convert to datetime, extract features
 - escalated: Binary feature (Yes/No)
 - sla_breached: Binary target for classification
 - operating_system/browser: Categorical features
 - payment_method: Categorical feature
 - language: Categorical feature
 - preferred_contact_time: Categorical feature
 - issue_complexity_score: Numerical feature
 - customer_segment: Categorical (Small Business, Corporate, Individual)
- Analyze data distribution and identify patterns using value counts and histograms
- Explore ticket categories and their characteristics:

- Compare priority distribution by category
- Analyze resolution time by category
- Check satisfaction scores by category

Phase 2: Data Preprocessing

- Handle missing values and outliers using median imputation for numerical features
- Convert data types (dates to datetime, clean and convert times to numeric)
- Text preprocessing for NLP tasks:
 - Lowercase text
 - Remove special characters and punctuation
 - Remove stopwords
 - Tokenization
 - Lemmatization or stemming
- Feature engineering:
 - Calculate ticket duration (resolved date - created date)
 - Create age groups (e.g., 18-25, 26-35, 36-50, 50+)
 - Extract temporal features (month, day of week, weekend flag)
 - Create ticket volume features by customer
 - Encode categorical variables using label encoding or one-hot encoding

Phase 3: Exploratory Data Analysis

- Univariate analysis: Distribution of age, satisfaction scores, resolution times, priority levels
- Bivariate analysis: Relationship between demographics and ticket characteristics
- Temporal analysis: Ticket patterns over time (seasonality, trends)
- Correlation analysis: Feature relationships
- Text analysis: Word clouds, frequent terms by category
- Visualization: Charts and graphs to communicate insights
 - Create at least 8-10 visualizations
 - Save all plots in folder
 - Add descriptive titles and labels to all plots

Phase 4: Feature Engineering & Selection

- Create new features from existing data
- Text feature extraction:
 - TF-IDF vectors for issue_description and resolution_notes
 - Word embeddings (Word2Vec, GloVe) or pre-trained models (BERT)
 - Text length, word count features
- Handle class imbalance using techniques like SMOTE, ADASYN, or class weights
- Split data into train/validation/test sets
- Scale numerical features
- Select important features using techniques like RFE, feature importance, chi-square

Phase 5: Model Building

- **Baseline Models:** Logistic Regression, Naive Bayes, Decision Tree
- **Advanced Models:** Random Forest, Gradient Boosting, XGBoost
- **Deep Learning Models** (Mandatory for at least one task):
 - **MLP/Dense Networks:** For tabular data classification and regression
 - **RNN Models:** LSTM, GRU for sequential text data
 - **Transformer Models:** BERT, RoBERTa for text classification
 - **CNN for Text:** Alternative approach for text classification
- **Ensemble Methods:** Combine multiple models for better performance
- **Comparison:** Compare deep learning performance with traditional ML models

Phase 6: Model Evaluation

- Use appropriate metrics for classification: Accuracy, Precision, Recall, F1-Score, ROC-AUC
- Use appropriate metrics for regression: R^2 , RMSE, MAE
- Perform cross-validation
- Tune hyperparameters using GridSearch or RandomSearch
- Analyze confusion matrix for classification tasks

Phase 7: Model Interpretation

- Feature importance analysis
- SHAP values for model explainability
- Error analysis: Understand misclassifications and prediction errors
- Text analysis: Identify important words/phrases for text classification

Phase 8: Deployment

- Save the trained model
- Build a basic UI for ticket analysis and prediction (Streamlit recommended)

ML Pipeline for Training, Testing, and Inference

Training Pipeline

Build a reusable training pipeline that includes:

- **Data Loading:** Load and validate data from CSV files
- **Preprocessing Pipeline:** Combine all preprocessing steps (scaling, encoding, text processing, feature engineering)
- **Model Training:** Train models with consistent hyperparameters
- **Validation:** Use cross-validation to ensure model robustness
- **Model Persistence:** Save trained models and preprocessing artifacts

The training pipeline should be modular so it can be reused across different problem statements.

Testing Pipeline

Create a testing pipeline that ensures consistent evaluation:

- **Test Data Loading:** Load test data separately from training data
- **Preprocessing:** Apply the same preprocessing steps used in training
- **Prediction:** Generate predictions on test data
- **Evaluation Metrics:** Calculate and report all relevant metrics
- **Error Analysis:** Analyze misclassifications and prediction errors
- **Reporting:** Generate comprehensive evaluation reports

Inference Pipeline

Build an inference pipeline for real-time predictions:

- **Input Validation:** Validate input data format and values
- **Preprocessing:** Apply the same preprocessing used during training
- **Model Loading:** Load trained models and preprocessing artifacts
- **Prediction:** Generate predictions on new data
- **Post-processing:** Convert model outputs to interpretable results
- **Output Formatting:** Return results in user-friendly format

Pipeline Implementation Approach

1. **Use Scikit-learn Pipeline:** Combine preprocessing and model training steps
2. **Custom Pipeline Classes:** Create custom transformers for feature engineering and text processing
3. **Pipeline Serialization:** Save entire pipeline (preprocessing + model) for inference
4. **Consistent Interface:** Ensure all pipelines have consistent input/output formats
5. **Error Handling:** Add try-except blocks for pipeline failures
6. **Logging:** Log pipeline steps for debugging and monitoring

Pipeline Benefits

- **Reproducibility:** Same preprocessing applied consistently
- **Modularity:** Easy to modify individual pipeline steps
- **Scalability:** Pipelines can be deployed as services
- **Maintainability:** Clear separation of concerns
- **Testing:** Individual pipeline components can be unit tested

Streamlit UI Development

UI Requirements

Build a Streamlit application that allows users to:

1. Input Data Entry:

- Ticket details (category, priority, channel)
- Customer demographics (age, gender, subscription type)
- Issue description (text input)
- Resolution notes (text input)

2. Prediction Display:

- Ticket priority prediction with probability
- Ticket category prediction
- Resolution time prediction
- Customer satisfaction prediction
- SLA breach risk assessment
- Color-coded results (green for low risk, red for high risk)

3. Additional Features:

- Load sample data button for quick testing
- Display model performance metrics
- Show feature importance chart
- Provide explanation for predictions
- Batch prediction option for multiple tickets
- Export results functionality

Streamlit App Structure

The Streamlit app should include:

- Page configuration with title and icon
- Sidebar for model selection and settings
- Main area for input and results
- Loading state for predictions
- Error handling for invalid inputs
- Responsive design for different screen sizes

Deployment Steps

1. Create `app.py` file with Streamlit code
2. Load trained model and preprocessing artifacts
3. Build input interface with Streamlit widgets
4. Implement prediction function
5. Add visualization for results
6. Test the app locally using `streamlit run app.py`
7. Deploy to Streamlit Cloud or other hosting platform

Results

Learners should aim to achieve:

For Ticket Priority Classification (Required):

- **Priority Classification Model:** Accuracy of 80%+ on test set for multi-class classification (Urgent, High, Medium, Low) using deep learning

For Additional Problem Statements (Choose at least 1 more):

- **Resolution Time Prediction:** R^2 score of 0.70+ for regression model using neural networks
- **Customer Satisfaction Prediction:** 75%+ accuracy for satisfaction classification or R^2 0.65+ for regression using neural networks
- **SLA Breach Prediction:** 85%+ accuracy for binary classification using neural networks

Overall Deliverables:

- **Comprehensive EDA Report:** Insights about ticket patterns, customer segments, resolution times, and satisfaction scores
- **Feature Importance:** Identification of key factors for each problem statement
- **Model Interpretability:** Clear explanation of model decisions across all models
- **Comparative Analysis:** Comparison of deep learning vs traditional ML models

Project Evaluation Metrics

Model Performance Metrics

- **Accuracy:** Overall correctness of predictions
- **Precision:** Out of predicted positive cases, how many were actually positive
- **Recall:** Out of actual positive cases, how many were correctly predicted
- **F1-Score:** Harmonic mean of precision and recall
- **ROC-AUC:** Area under the ROC curve (for binary classification)
- **Confusion Matrix:** Breakdown of true/false positives/negatives
- **R^2 Score:** Coefficient of determination (for regression)

- **RMSE:** Root Mean Square Error (for regression)
- **MAE:** Mean Absolute Error (for regression)

Project Evaluation Criteria

1. **Code Quality:** Clean, modular, and well-documented code
2. **Data Understanding:** Thorough EDA with meaningful insights
3. **Feature Engineering:** Creative and relevant feature creation
4. **Model Performance:** Achieving target metrics
5. **Documentation:** Clear README and code comments
6. **NLP Implementation:** Proper text preprocessing and feature extraction
7. **Pipeline Implementation:** End-to-end ML pipeline for training, testing, and inference

Technical Tags

- Python
- Pandas
- NumPy
- Scikit-learn
- TensorFlow/Keras
- PyTorch
- NLP (NLTK, spaCy, or TextBlob)
- Deep Learning
- RNN (LSTM, GRU)
- Transformer Models (BERT, RoBERTa)
- Text Preprocessing
- Text Embeddings (Word2Vec, GloVe, BERT)
- Feature Engineering
- Machine Learning
- Classification
- Regression
- Imbalanced Learning
- Model Evaluation
- SHAP
- Streamlit
- Data Visualization
- Matplotlib
- Seaborn

Data Set

Dataset Source

Customer Support Tickets Dataset (200K+ Records) - A large-scale, structured, and text-rich dataset designed for NLP, machine learning classification, and customer support analytics tasks.

Dataset Files

- 1. **customer_support_ticket.csv** - Primary dataset with 200,000+ customer support tickets. Supports all problem statements.

Recommended Dataset

Use customer_support_ticket.csv as the primary dataset for all problem statements. Use train/validation/test splits (e.g., 70%/15%/15%) for model evaluation.

Dataset Columns (customer_support_ticket.csv)

Column	Description	Data Type
ticket_id	Unique identifier for each ticket	Integer
customer_name	Customer name	String
customer_email	Customer email	String
product	Product/Service (Web Portal, Mobile App, Payment Gateway, etc.)	String
category	Ticket category (10 categories)	String
issue_description	Text description of the issue	String
resolution_notes	Text notes on resolution	String
priority	Priority level (Urgent, High, Medium, Low)	String
status	Ticket status (Open, Closed, Pending Customer, In Progress, Resolved)	String
channel	Communication channel (Email, Chat, Social Media, Phone)	String

Column	Description	Data Type
region	Geographic region	String
customer_age	Customer age	Integer
customer_gender	Customer gender	String
subscription_type	Subscription type (Free, Premium, Basic, Enterprise)	String
customer_tenure_months	Customer tenure in months	Integer
previous_tickets	Number of previous tickets	Integer
customer_satisfaction_score	Satisfaction score (1-5)	Integer
first_response_time_hours	Time to first response in hours	Float
resolution_time_hours	Time to resolution in hours	Float
ticket_created_date	Date ticket was created	Date
ticket_resolved_date	Date ticket was resolved	Date
escalated	Whether ticket was escalated (Yes/No)	String
sla_breached	Whether SLA was breached (Yes/No)	String
operating_system	Customer's operating system	String
browser	Customer's browser	String
payment_method	Payment method used	String
language	Customer's language	String
preferred_contact_time	Preferred contact time	String
issue_complexity_score	Complexity score (1-10)	Integer
customer_segment	Customer segment (Small Business, Corporate, Individual)	String

Dataset Statistics

- **Total Records:** 200,000+ tickets
- **Total Features:** 30+
- **Text Fields:** issue_description, resolution_notes
- **Categories:** 10 ticket categories (Login Issue, Payment Problem, Account Suspension, Bug Report, Feature Request, Performance Issue, Refund Request, Subscription Cancellation, Security Concern, Data Sync Issue)
- **Priority Levels:** 4 (Urgent, High, Medium, Low)
- **Customer Segments:** 3 (Small Business, Corporate, Individual)

Data Preprocessing Requirements

- Handle missing values in text fields and numerical columns
- Convert date columns to datetime format
- Clean and preprocess text fields for NLP tasks
- Encode categorical variables (priority, status, channel, region, etc.)
- Scale numerical features for regression tasks
- Handle class imbalance in target variables (priority, category, satisfaction score)

Data Set Explanation

The customer support tickets dataset contains 200,000+ realistic customer support tickets across multiple products and service domains. Each ticket represents a customer interaction with a support team and includes both structured metadata and unstructured text data.

Content Overview:

The dataset captures the complete lifecycle of customer support tickets from creation to resolution. Each record includes customer demographics (age, gender, subscription type, tenure), ticket metadata (priority, status, channel, product), operational metrics (response time, resolution time, SLA compliance), and rich text fields (issue description, resolution notes).

Context and Use Case:

This dataset simulates real-world customer support scenarios for a multi-product SaaS company offering services through various channels (Web Portal, Mobile App, Payment Gateway, API Service, etc.). The data reflects actual customer pain points including login issues, payment problems, account suspensions, bug reports, feature requests, and security concerns.

Data Characteristics:

- **Structured Data:** 28+ categorical and numerical features including customer demographics, product information, subscription details, and operational metrics

- **Unstructured Data:** Two text fields (issue_description and resolution_notes) that capture the customer's problem and the support team's solution in natural language
- **Temporal Data:** Ticket creation and resolution dates enabling time-series analysis
- **Multi-dimensional:** Tickets span multiple regions, languages, operating systems, and customer segments (Individual, Small Business, Corporate)
- **Imbalanced Classes:** Priority levels and ticket categories have imbalanced distributions, reflecting real-world scenarios where urgent tickets are less frequent

Preprocessing Context:

The dataset requires comprehensive preprocessing including:

- Text cleaning and normalization for NLP tasks
- Categorical encoding for machine learning models
- Feature engineering to create derived variables (ticket duration, customer behavior metrics)
- Handling of missing values and outliers
- Text tokenization and embedding generation for deep learning models

This dataset is ideal for practicing end-to-end machine learning workflows combining tabular data with NLP techniques, making it suitable for both traditional ML and deep learning approaches.

Project Deliverables

1. Source Code

- Well-structured Python code with clear function definitions
- Separate modules for data loading, preprocessing, feature engineering, model training, and evaluation
- Model training and evaluation (separate modules for each problem statement)
- Utilities and helper functions

2. Notebooks

- Detailed notebooks for each problem statement with step-by-step analysis
- Markdown cells explaining each step
- Visualizations and insights from EDA

3. Trained Models

- Saved model files for each problem statement
- Model performance metrics on test set for each model
- Feature importance scores for each model

4. Documentation

- README.md with project description, setup instructions, and usage guide

- Requirements.txt with all dependencies
- Brief report summarizing findings and model performance

Guidelines

Project Structure

Organize your project as follows:

customer_support/

|— data/

| |— customer_support_ticket.csv

|— notebooks/

| |— 01_eda.ipynb

| |— 02_priority_classification.ipynb

| |— 03_category_classification.ipynb

| |— 04_resolution_time_prediction.ipynb

|— src/

| |— data_loader.py

| |— preprocessing.py

| |— feature_engineering.py

| |— models.py

| |— utils.py

|— models/

| |— priority_classifier.pkl

| |— category_classifier.pkl

| |— resolution_time_regressor.pkl

|— app.py

└─ requirements.txt

└─ README.md

Best Practices

1. **Code Organization:** Use modular code with clear separation of concerns
2. **Variable Naming:** Use descriptive variable names
3. **Comments:** Add comments to explain complex logic
4. **Error Handling:** Include try-except blocks for file operations
5. **Modular Code:** Break down code into reusable functions

Project Structure

Best Practices

1. **Reproducibility:** Set random seeds for reproducible results
2. **Data Leakage Prevention:** Ensure no data leakage between train and test sets
3. **Cross-Validation:** Use cross-validation for robust model evaluation
4. **Hyperparameter Tuning:** Tune hyperparameters systematically
5. **Model Interpretation:** Provide explanations for model predictions
6. **Documentation:** Document all assumptions and decisions

Common Pitfalls & Troubleshooting

1. **Data Leakage:** Ensure preprocessing is fit only on training data
2. **Class Imbalance:** Use appropriate techniques to handle imbalanced classes
3. **Overfitting:** Use regularization and cross-validation to prevent overfitting
4. **Text Processing:** Ensure consistent text preprocessing across train and test sets
5. **Memory Issues:** For large datasets, use chunking or sampling for EDA
6. **NLP Challenges:** Handle out-of-vocabulary words and rare terms appropriately

Milestone

- **Milestone 1:** Data understanding, EDA, and initial preprocessing
- **Milestone 2:** Feature engineering and baseline model development
- **Milestone 3:** Advanced model training and hyperparameter tuning
- **Milestone 4:** Model evaluation, interpretation, and documentation
- **Milestone 5:** Streamlit UI development and deployment